**FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA**

Gabriel Neris Losano, RM564093, 2TDSPG

Joao Vitor Biribilli, RM565594, 2TDSPG

Pietro Paranhos Wilhelm, 561378, 2TDSPG

# PetPulse

São Paulo

2026

**FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA**

Gabriel Neris Losano, RM564093, 2TDSPG
Joao Vitor Biribilli, RM565594, 2TDSPG
Pietro Paranhos Wilhelm, 561378, 2TDSPG

# PetPulse

*Sprint* 4 apresentado à Faculdade de Informática e Administração Paulista como requisito de nota para a avaliação da disciplina Mastering Relational and Non-Relational Database, sob a orientação do Professor Vergílio Valério dos Santos

São Paulo
2026

# SUMÁRIO

1 **DESCRIÇAO DO PROJETO** ..... 4

1.1 Objetivo da Solução ..... 4

1.2 Justificativa da Escolha do Tema ..... 4

1.3 Tecnologias Utilizadas ..... 4

2 **Mastering Relational and Non-Relational Database** ..... 5

2.1 levantamento de requisitos.....6

2.2 modelagens descritiva.....6

2.3 modelos lógico e relacional.....7

2.4 3FN.....8

2.5 Prints de funcionamento.....9

2.6 Passo a passo de execução.....12

# 1 DESCRIÇAO DO PROJETO

## 1.1 Objetivo da Solução

O banco de dados do projeto Clyvo Vet tem como objetivo armazenar e organizar as informações necessárias para o acompanhamento preventivo da saúde de pets.

A estrutura foi criada para permitir o cadastro de tutores, pets, histórico clínico, dispositivos IoT e alertas inteligentes. Com esses dados, o sistema poderá apoiar a continuidade do cuidado veterinário, registrando informações importantes do animal e permitindo a geração de alertas preventivos com base em dados clínicos e dados coletados por dispositivos.

O modelo foi pensado para o escopo inicial do MVP, mantendo uma estrutura simples, relacional e coerente com a proposta do projeto.

## 1.2 Justificativa da Escolha do Tema

A escolha da **PetPulse** como tema deste trabalho nasce da observação de um problema concreto e cotidiano: tutores de pets não contam com ferramentas acessíveis para monitorar a saúde dos seus animais de forma contínua. A maioria só percebe que algo está errado quando o pet já apresenta sintomas avançados, e nesse ponto o tratamento tende a ser mais caro, mais agressivo e com menos chance de sucesso. O tutor acaba se sentindo impotente, pois não tem como saber o que está acontecendo com o animal nos momentos em que não está em casa ou quando o pet simplesmente não consegue demonstrar que algo não vai bem — e os animais, por instinto, costumam esconder sinais de fraqueza. A PetPulse propõe uma solução simples na essência, mas apoiada em tecnologia: um aplicativo onde o tutor cria perfis completos para cada pet, registrando dados de saúde ao longo do tempo. Quando esses dados apontam alguma instabilidade — como alterações nos batimentos cardíacos — o sistema dispara um alerta e recomenda uma consulta veterinária. O verdadeiro diferencial está na coleira inteligente, que funciona como um smartwatch, capturando sinais vitais em tempo real e enviando-os diretamente ao aplicativo, eliminando a necessidade de o tutor inserir dados manualmente. Isso resolve um problema enorme de adesão: soluções que dependem do usuário lembrar de registrar informações todos os dias simplesmente não funcionam na prática. Com a coleta automatizada dos dados, o aplicativo consegue construir um histórico completo da saúde do pet ao longo do tempo, identificando padrões e tendências que passariam despercebidos em consultas eventuais. Uma alteração sutil nos batimentos cardíacos ao longo de semanas, por exemplo, pode indicar um problema cardíaco incipiente — e o sistema consegue capturar isso antes que se torne uma emergência. É a diferença entre esperar o pet ficar doente e agir preventivamente para manter a saúde em dia. O Brasil tem hoje mais de 140 milhões de animais de estimação, e a maioria dos tutores trata seus pets como membros da família. Mesmo assim, o acompanhamento veterinário ainda é majoritariamente reativo. A PetPulse entra exatamente nesse gap: oferece uma ferramenta que qualquer tutor pode usar no dia a

dia, sem precisar de conhecimento técnico, para ter mais tranquilidade sobre a saúde do seu animal. A coleira inteligente faz todo o trabalho pesado de coleta, e o aplicativo traduz esses dados em informações claras e acionáveis — como um alerta dizendo "hora de marcar uma consulta".

## 1.3 Tecnologias Utilizadas

Oracle Data Modeler

Oracle Developer

# 2. BUILDING RELATIONAL DATABASE

## 2.1 Levantamento de requisitos

É a identificação das informações essenciais que precisam ser armazenadas para que a solução funcione. É onde você descreve quais entidades existem no seu sistema e quais dados são importantes para cada uma.

## 2.2 Modelagem descritiva

T_CLY_USUARIO

Armazena os dados do tutor/responsável pelo pet.

T_CLY_PET

Armazena os dados cadastrais do pet e se relaciona com o tutor.

T_CLY_HISTORICO_CLINICO

Armazena registros como vacinas, consultas, doenças, medicamentos, observações e exames.

T_CLY_DISPOSITIVO_IOT

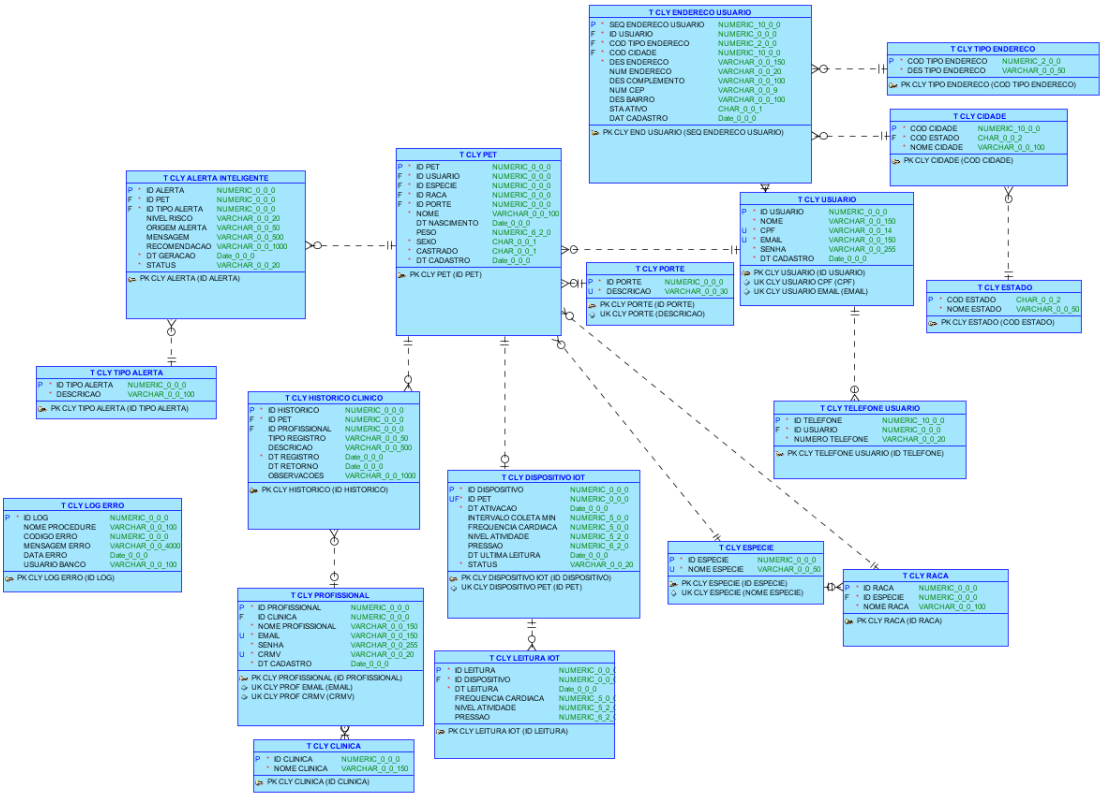
Armazena o dispositivo vinculado ao pet e a leitura atual consolidada.

T_CLY_ALERTA_INTELIGENTE

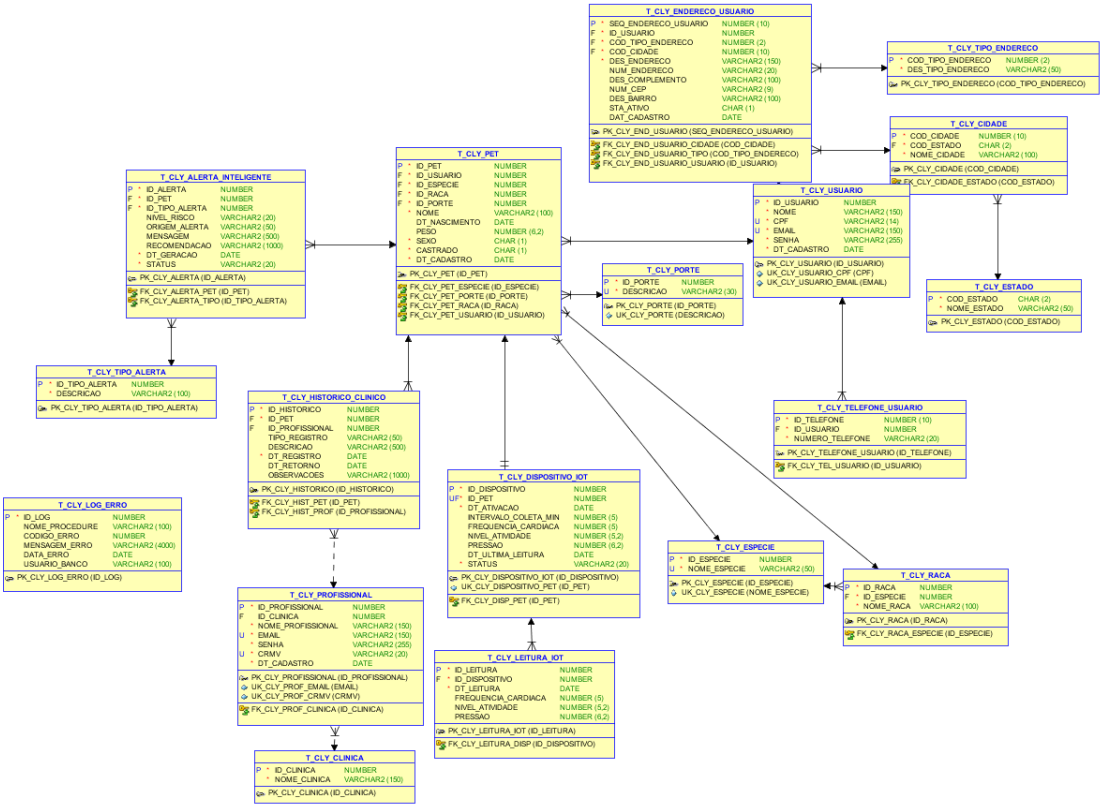
Armazena os alertas gerados pelo sistema.

## 2.3 Modelagem Lógica e Relational

Modelo Logical:




Modelo Relational:



## 2.4 3FN

O modelo foi estruturado para atender à **Terceira Forma Normal**, conforme exigido na disciplina de banco.

### Primeira Forma Normal — 1FN

O modelo atende à 1FN porque todos os atributos armazenam valores atômicos, ou seja, cada campo guarda apenas uma informação por registro.

Exemplos:

- `nome` armazena apenas o nome.
- `cpf` armazena apenas o CPF.
- `email` armazena apenas o e-mail.
- `especie` armazena apenas a espécie.
- `tipo_registro` armazena apenas o tipo do registro clínico.
- `nivel_risco` armazena apenas o nível de risco.

Não existem colunas com listas, grupos repetidos ou múltiplos valores no mesmo campo.

### Segunda Forma Normal — 2FN

O modelo atende à 2FN porque todas as tabelas possuem chave primária simples, e todos os atributos não-chave dependem integralmente da chave primária da própria tabela.

Exemplos:

Na tabela `T_CLY_USUARIO`, os campos `nome`, `cpf`, `email`, `telefone` e `endereco` dependem diretamente de `id_usuario`.

Na tabela `T_CLY_PET`, os campos `nome`, `especie`, `raca`, `peso`, `sexo`, `castrado` e `porte` dependem diretamente de `id_pet`.

Na tabela `T_CLY_ALERTA_INTELIGENTE`, os campos `tipo_alerta`, `nivel_risco`, `mensagem`, `recomendacao`, `dt_geracao` e `status` dependem diretamente de `id_alerta`.

---

**Terceira Forma Normal — 3FN**

O modelo atende à 3FN porque não há dependências transitivas relevantes entre os atributos não-chave.

Os dados foram separados por responsabilidade:

- Dados do tutor ficam em `T_CLY_USUARIO`.
- Dados do pet ficam em `T_CLY_PET`.
- Dados clínicos ficam em `T_CLY_HISTORICO_CLINICO`.
- Dados do dispositivo ficam em `T_CLY_DISPOSITIVO_IOT`.
- Dados dos alertas ficam em `T_CLY_ALERTA_INTELIGENTE`.

As tabelas filhas não repetem dados das tabelas pai. Elas armazenam apenas as chaves estrangeiras necessárias.

Exemplo:

A tabela `T_CLY_ALERTA_INTELIGENTE` não repete o nome do pet, o nome do tutor ou o e-mail do tutor. Ela armazena apenas `id_pet`. Quando essas informações forem necessárias, podem ser obtidas por meio de `JOIN`.

Da mesma forma, a tabela `T_CLY_HISTORICO_CLINICO` não repete os dados cadastrais do pet. Ela referência o pet por meio de `id_pet`.

Isso reduz redundância, evita inconsistências e mantém o modelo organizado.

## 2.5 Prints de funcionamento

Prints dos relatórios:

Parte 1

```
=== RELATORIO: PETS E SEUS ALERTAS ===
Pet: Rex | Alerta: FREQUENCIA_CARDIACA | Risco: ALTO | Prioridade: P1 - URGENTE
Pet: Thor | Alerta: PRESSAO | Risco: ALTO | Prioridade: P1 - URGENTE
Pet: Bolinha | Alerta: ATIVIDADE | Risco: MEDIO | Prioridade: P2 - MODERADA
Pet: Mel | Alerta: VACINA | Risco: MEDIO | Prioridade: P2 - MODERADA
Pet: Luna | Alerta: ALIMENTACAO | Risco: BAIXO | Prioridade: P3 - BAIXA


Procedimento PL/SQL concluido com sucesso.

=== RELATORIO: REGISTROS CLINICOS POR ESPECIE ===
Especie: CACHORRO | Total registros: 5
Especie: GATO | Total registros: 1


Procedimento PL/SQL concluido com sucesso.

=== RELATORIO 8: CONSOLIDADO GERAL (SUMARIZACAO) ===
--- Usuarios e Pets ---
Total de usuarios cadastrados : 5
Total de pets cadastrados     : 7
Peso medio dos pets (kg)      : 15,57
--- Dispositivos IoT ---
Total de dispositivos         : 5
  Ativos      : 3
  Inativos    : 1
  Manutencao  : 1
Total de leituras registradas : 14
--- Historico Clinico ---
Total de registros clinicos   : 6
--- Alertas Inteligentes ---
Total de alertas              : 5
  Em aberto       : 3 (60%)
  Alto risco ativo : 2
  Resolvidos      : 1
--- Situacao Geral do Sistema ---
Diagnostico: CRITICA - existem alertas de alto risco nao resolvidos


Procedimento PL/SQL concluido com sucesso.
```

Parte 2

```
=== RELATORIO: PESO ANTERIOR / ATUAL / PROXIMO ===
Pet: Rex | Anterior: Vazio | Atual: 28 | Proximo: 4
Pet: Mel | Anterior: 28 | Atual: 4 | Proximo: 30
Pet: Thor | Anterior: 4 | Atual: 30 | Proximo: 5
Pet: Luna | Anterior: 30 | Atual: 5 | Proximo: 12
Pet: Bolinha | Anterior: 5 | Atual: 12 | Proximo: 15
Pet: Pet Auditoria Teste | Anterior: 12 | Atual: 15 | Proximo: 15
Pet: Pet Auditoria Teste | Anterior: 15 | Atual: 15 | Proximo: Vazio


Procedimento PL/SQL concluido com sucesso.

=== RELATORIO: USUARIOS CADASTRADOS ===
Nome: Armando Souza | Email: armando@email.com
Nome: Beatriz Costa | Email: beatriz@email.com
Nome: Carlos Pereira | Email: carlos@email.com
Nome: Daniela Martins | Email: daniela@email.com
Nome: Eduardo Ramos | Email: eduardo@email.com


Procedimento PL/SQL concluido com sucesso.

=== RELATORIO: PETS CADASTRADOS ===
Pet: Bolinha | Especie: CACHORRO | Porte: MEDIO | Peso: 12 | Diagnostico: Peso compativel com o porte
Pet: Luna | Especie: CACHORRO | Porte: PEQUENO | Peso: 5 | Diagnostico: Peso compativel com o porte
Pet: Mel | Especie: GATO | Porte: PEQUENO | Peso: 4 | Diagnostico: Peso compativel com o porte
Pet: Pet Auditoria Teste | Especie: CACHORRO | Porte: MEDIO | Peso: 15 | Diagnostico: Peso compativel com o porte
Pet: Pet Auditoria Teste | Especie: CACHORRO | Porte: MEDIO | Peso: 15 | Diagnostico: Peso compativel com o porte
Pet: Rex | Especie: CACHORRO | Porte: GRANDE | Peso: 28 | Diagnostico: Peso compativel com o porte
Pet: Thor | Especie: CACHORRO | Porte: GRANDE | Peso: 30 | Diagnostico: Peso compativel com o porte


Procedimento PL/SQL concluido com sucesso.

=== RELATORIO: HISTORICO CLINICO ===
Tipo: VACINA | Descricao: Vacina antirabica aplicada | Profissional: Dr. Carlos Andrade | Clinica: Clinica VetCare | Situacao Retorno: RETORNO ATRASADO
Tipo: CONSULTA | Descricao: Consulta de rotina - saudavel | Profissional: Dr. Carlos Andrade | Clinica: Clinica VetCare | Situacao Retorno: RETORNO ATRASADO
Tipo: DOENCA | Descricao: Diagnostico de leishmaniose | Profissional: Dr. Roberto Souza | Clinica: Hospital Veterinario PetSaude | Situacao Retorno: RETORNO ATRASADO
Tipo: EXAME | Descricao: Hemograma completo solicitado | Profissional: Dra. Fernanda Lima | Clinica: Clinica VetCare | Situacao Retorno: RETORNO ATRASADO
Tipo: CONSULTA | Descricao: Avaliacao pos-operatoria | Profissional: Dra. Fernanda Lima | Clinica: Clinica VetCare | Situacao Retorno: RETORNO ATRASADO
Tipo: MEDICAMENTO | Descricao: Vermifugo aplicado - dose mensal | Profissional: Dr. Carlos Andrade | Clinica: Clinica VetCare | Situacao Retorno: Sem retorno agendado
```

Print dos Triggers:

Exemplo de erro



```
Erro: Porte ja existente
Erro: Porte ja existente


Procedimento PL/SQL concluido com sucesso.

Leitura IoT registrada com sucesso
Leitura IoT registrada com sucesso
Leitura IoT registrada com sucesso


Procedimento PL/SQL concluido com sucesso.


Function FUNC_PET_TO_JSON compilado


Function FUNC_VALIDA_PESO_PORTE compilado


Procedure PRC_REL_PETS_JSON compilado


Procedure PRC_REL_LEITURAS_SUBTOTAL compilado


Procedimento PL/SQL concluido com sucesso.


Table T_CLV_AUDITORIA_PET criado.

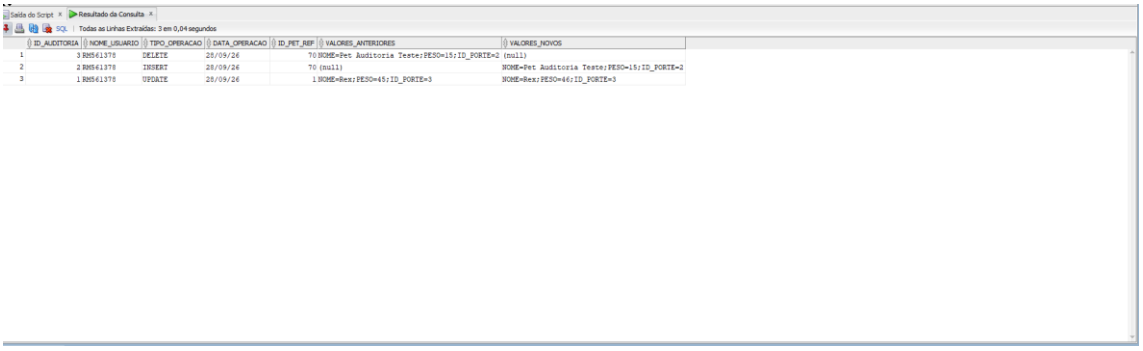

Trigger TRG_AUDITORIA_PET compilado

Teste Funcao 1 (sucesso): {"idPet":1,"nomePet":"Rex","tutor":"Armando Souza","especie":"CACHORRO","raca":"Golden Retriever","porte":"GRANDE","peso":25,"sexo":"M"}


Procedimento PL/SQL concluido com sucesso.

Teste Funcao 1 (erro esperado): ORA-20100: FUNC_PET_TO_JSON: id do pet invalido (nulo ou <= 0)
```

Exemplo das auditorias



Saída do Script | Resultado da Consulta

Todas as Linhas Extraídas: 3 em 0,04 segundos

| | ID_AUDITORIA | NOME_USUARIO | TIPO_OPERACAO | DATA_OPERACAO | ID_PET_REF | VALORES_ANTERIORES | VALORES_NOVOS |
|---|---|---|---|---|---|---|---|
| 1 | 3 | RM561378 | DELETE | 28/09/26 | 70 | NOME=Pet Auditoria Teste;PESO=15;ID_PORTE=2 | (null) |
| 2 | 2 | RM561378 | INSERT | 28/09/26 | 70 | (null) | NOME=Pet Auditoria Teste;PESO=15;ID_PORTE=2 |
| 3 | 1 | RM561378 | UPDATE | 28/09/26 | 1 | NOME=Rex;PESO=45;ID_PORTE=3 | NOME=Rex;PESO=46;ID_PORTE=3 |

Dados em formato Json



```
Teste Funcao 2 (erro esperado): ORA-20111: FUNC_VALIDA_PESO_PORTE: porte invalido - use PEQUENO, MEDIO, GRANDE, MINI ou GIGANTE


Procedimento PL/SQL concluido com sucesso.

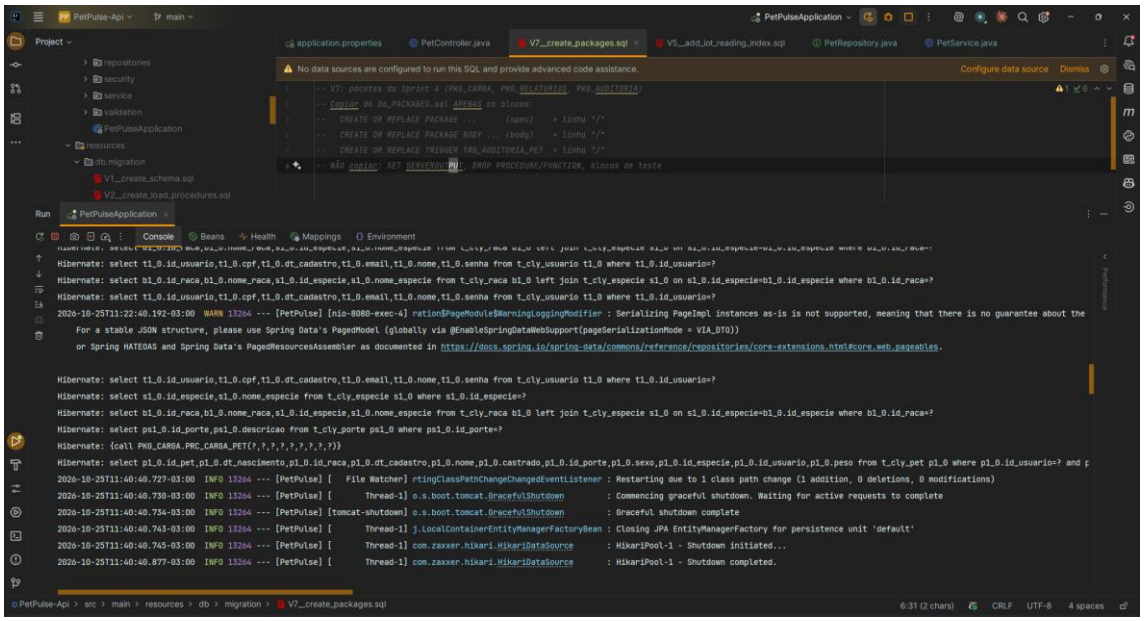
=== PROCEDIMENTO 1: PETS EM FORMATO JSON (JOIN MANUAL) ===
{"idPet":1,"nomePet":"Rex","tutor":"Armando Souza","especie":"CACHORRO","raca":"Golden Retriever","porte":"GRANDE","peso":27,"sexo":"M"}
{"idPet":2,"nomePet":"Mel","tutor":"Beatriz Costa","especie":"GATO","raca":"Siames","porte":"PEQUENO","peso":4,"sexo":"F"}
{"idPet":3,"nomePet":"Thor","tutor":"Carlos Pereira","especie":"CACHORRO","raca":"Labrador","porte":"GRANDE","peso":30,"sexo":"M"}
{"idPet":4,"nomePet":"Luna","tutor":"Daniela Martins","especie":"CACHORRO","raca":"Shih Tzu","porte":"PEQUENO","peso":5,"sexo":"F"}
{"idPet":5,"nomePet":"Bolinha","tutor":"Eduardo Ramos","especie":"CACHORRO","raca":"Bulldog","porte":"MEDIO","peso":12,"sexo":"M"}
{"idPet":21,"nomePet":"Pet Auditoria Teste","tutor":"Armando Souza","especie":"CACHORRO","raca":"Golden Retriever","porte":"MEDIO","peso":15,"sexo":"M"}
--- Array JSON completo (6 pets) ---
[{"idPet":1,"nomePet":"Rex","tutor":"Armando Souza","especie":"CACHORRO","raca":"Golden Retriever","porte":"GRANDE","peso":27,"sexo":"M"},{"idPet":2,"nomePet":"Mel","tutor":"Beatriz Costa","especie":"GATO","raca":"Siames",

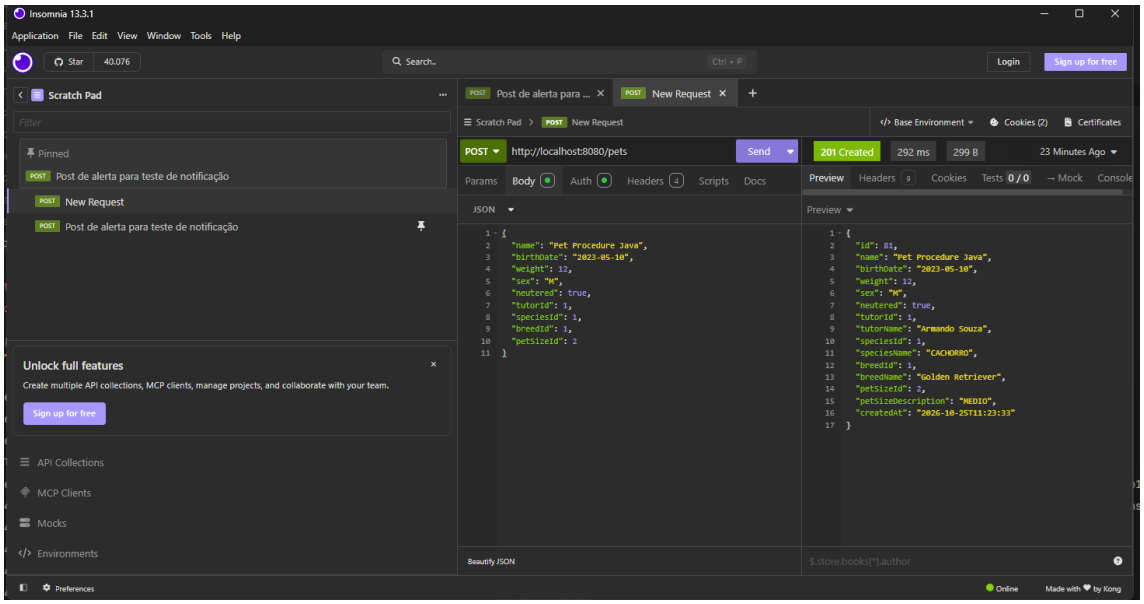

Procedimento PL/SQL concluido com sucesso.

=== PROCEDIMENTO 2: LEITURAS IOT COM SUBTOTAL POR DISPOSITIVO ===
Dispositivo  Dia         BPM
-----------  ----------  -----
1            24/08/2026    78
1            25/08/2026    78
1            26/08/2026    80
1            27/08/2026    85
1            31/08/2026    78
             Sub Total     399
2            25/08/2026   140
2            26/08/2026   155
2            27/08/2026   155
2            02/09/2026   155
             Sub Total     625
3            25/08/2026    70
3            26/08/2026    70
3            26/08/2026    72
3            27/08/2026    75
3            01/09/2026    70
             Sub Total     357
             Total Geral   1381


Procedimento PL/SQL concluido com sucesso.
```

Procedure sendo chamada no backend – referente a criação de um PET



### 2.6 Passo a passo de execução

Esta seção descreve como reproduzir toda a entrega do zero: criação do banco relacional, empacotamento das procedures, funções e gatilho, exportação das leituras IoT para JSON, importação no MongoDB e a chamada das procedures pela API Java.

**Pré-requisitos**

– **Oracle SQL Developer** conectado ao schema do projeto (ex.: RM561378).

– **MongoDB** (servidor local) e **MongoDB Compass**.

– **MongoDB Database Tools**, que fornece o comando mongoimport. É instalado separadamente do MongoDB.

– **API PetPulse** (Java Spring Boot) aberta no IntelliJ, com JDK configurado.

**Ordem de execução dos arquivos**

| # | Arquivo | Onde rodar | O que faz |
|---|---|---|---|
| 1 | 00_DROP.sql | SQL Developer | Remove pacotes, procedures, trigger e tabelas (reset completo). |
| 2 | 01_DDL.sql | SQL Developer | Cria as 18 tabelas T_CLY_* em 3FN. |
| 3 | 02_PROCEDURES.sql | SQL Developer | Cria as 17 procedures de carga PRC_CARGA_*. |
| 4 | 03_CARGA.sql | SQL Developer | Popula as tabelas chamando as procedures de carga. |
| 5 | 04_RELATORIOS.sql | SQL Developer | Relatórios com cursores, JOIN, GROUP BY e relatório consolidado. |
| 6 | 05_SPRINT3.sql | SQL Developer | Funções, procedures e trigger de auditoria da Sprint 3, com testes. |
| 7 | 06_PACKAGES.sql | SQL Developer | Cria PKG_CARGA, PKG_RELATORIOS e PKG_AUDITORIA e remove os objetos avulsos. |
| 8 | exporta_json.sql | SQL Developer | Cria PKG_EXPORTACAO e gera o arquivo leituras_iot.json. |
| 9 | 08_MONGODB_ESTRUTURA.js | Compass (mongosh) | Cria a coleção leituras_iot com validador e índices. |
| 10 | mongoimport | PowerShell | Importa o leituras_iot.json para o MongoDB. |

| # | Arquivo | Onde rodar | O que faz |
|---|---|---|---|
| 11 | 09_MONGODB_CONSULTAS.js | Compass (mongosh) | Consultas de verificação e teste do validador. |

**Atenção:** o 05_SPRINT3.sql deve rodar antes do 06_PACKAGES.sql. Depois do 06, as procedures avulsas deixam de existir e só podem ser chamadas pelo pacote (ex.: PKG_CARGA.PRC_CARGA_PET).

**Passo 1 – Banco relacional (Oracle)**

Feche as abas antigas do projeto no SQL Developer, abra cada arquivo direto do disco e execute com F5 (Executar Script), na ordem da tabela acima (arquivos 1 a 7). Acompanhe o resultado no painel Saída do Script, com SET SERVEROUTPUT ON habilitado. Ao final, confira se os objetos estão válidos:

```
SELECT object_name, object_type, status
FROM user_objects
WHERE object_name LIKE 'PKG_%' OR object_name = 'TRG_AUDITORIA_PET'
ORDER BY object_name, object_type;
```

Todos os objetos devem aparecer com status VALID.

**Passo 2 – Exportação das leituras IoT para JSON**

Abra o exporta_json.sql e execute com F5. O bloco de SPOOL no final do script grava o arquivo leituras_iot.json na pasta E:\2TDSPG\challenge\PetPulseDB, com um documento por leitura. Anote a quantidade de leituras para comparar com o MongoDB:

```
SELECT COUNT(*) FROM T_CLY_LEITURA_IOT;
```

**Passo 3 – Estrutura no MongoDB**

No MongoDB Compass, conecte em mongodb://localhost:27017, abra o terminal mongosh embutido (aba _MONGOSH) e cole o conteúdo do 08_MONGODB_ESTRUTURA.js. Ele usa o banco petpulse e cria a coleção leituras_iot com validação por $jsonSchema e os índices idx_pet_data e idx_dispositivo_data.

**Passo 4 – Importação do JSON**

No PowerShell, entre na pasta do arquivo e execute o mongoimport. Como a pasta do MongoDB Database Tools normalmente não está no PATH do Windows, use o caminho completo do executável precedido de & (ajuste a pasta da versão instalada):

```
cd E:\2TDSPG\challenge\PetPulseDB

& "C:\Program Files\MongoDB\Tools\100\bin\mongoimport.exe" `
    --db petpulse --collection leituras_iot `
    --file leituras_iot.json --jsonArray --mode upsert
```

Resultado esperado: “16 document(s) imported successfully. 0 document(s) failed to import.” A opção --mode upsert permite repetir a importação sem duplicar documentos.

Para localizar o executável, ou instalá-lo caso não exista:

```
Get-ChildItem "C:\Program Files\MongoDB" -Recurse -Filter mongoimport.exe |
    Select-Object FullName

winget install MongoDB.DatabaseTools
```

**Passo 5 – Verificação no MongoDB**

No mongosh do Compass, cole o 09_MONGODB_CONSULTAS.js. Ele mostra as leituras por pet, as leituras com frequência cardíaca acima de 140, a média, mínima e máxima por pet e, por fim, demonstra o validador rejeitando um documento inválido. A contagem deve ser igual à do Oracle:

```
use petpulse
db.leituras_iot.countDocuments()
```

**Passo 6 – Procedures chamadas pela API Java**

Na configuração de execução da API no IntelliJ (Edit Configurations → Environment variables), defina as variáveis de conexão com o Oracle:

```
SPRING_DATASOURCE_URL=jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL
SPRING_DATASOURCE_USERNAME=<seu RM>
SPRING_DATASOURCE_PASSWORD=<sua senha>
```

Ao iniciar, o Flyway aplica as migrations V1 a V7. A V7 cria os mesmos pacotes do 06_PACKAGES.sql, garantindo que a API funcione em qualquer schema. Em seguida:

– Acesse http://localhost:8080/swagger-ui/index.html.

– Faça login em POST /login (ex.: carlos.andrade@vetcare.com / 123456) e informe o token em Authorize.

– Cadastre um pet em POST /pets. O cadastro é feito pela procedure, via @Procedure no PetRepository.

– No console do IntelliJ aparece a chamada {call PKG_CARGA.PRC_CARGA_PET(?,?,?,?,?,?,?,?,?)}.

Confirme no SQL Developer o pet inserido e o registro gerado pela trigger de auditoria:

```
SELECT * FROM T_CLY_PET ORDER BY ID_PET DESC FETCH FIRST 1 ROWS ONLY;
SELECT * FROM T_CLY_AUDITORIA_PET ORDER BY 1 DESC FETCH FIRST 1 ROWS ONLY;
```

**Solução de problemas**

| Erro | Causa e solução |
|---|---|
| ORA-06502 nos blocos de carga | Aba antiga em cache no SQL Developer. Feche o programa, reabra o arquivo do disco e rode com F5. Use valores inteiros nos campos numéricos da carga. |
| ORA-00955: name is already used | Script de criação executado duas vezes. Rode o 00_DROP.sql antes. |
| PLS-00201 ao rodar o 05 depois do 06 | As procedures avulsas foram removidas pelo 06. Respeite a ordem 05 → 06 ou chame pelo pacote (PKG_CARGA.*). |
| mongoimport não é reconhecido | O MongoDB Database Tools não está no PATH ou não está instalado. Use o caminho completo com & ou instale com winget. |
| ORA-12514 ao iniciar a API | URL de conexão sem o SID. Use jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL. |
| Login retorna 401 após recarregar o banco | O 03_CARGA.sql grava senhas em texto puro; a API espera BCrypt. Reaplique o UPDATE de senhas da migration V6. |

Vídeo mostrando

adendo o youtube pode colocar o vídeo na qualidade mais baixa caso ocorra

coloque na mais alta disponivel por gentileza

Link:

https://youtu.be/Y94yoK8vFbg